Walmart Data Warehouse Project Report

Course: DS3003 - Data Warehousing & Business Intelligence
Semester: Fall 2025
Name: Muhammad Ahmad
Roll No: 22I-1929
Submission Date: November 17, 2025

Contents

1 Project Overview

1.1 Introduction

This project implements a near-real-time data warehouse (DW) prototype for Walmart, one of the world's largest retail chains. The system is designed to integrate and process transactional data efficiently, enabling timely business insights and dynamic decision-making for optimizing sales and enhancing customer satisfaction.

1.2 Project Objectives

The main objectives of this project are:

1. **Design and implement a star schema** for Walmart's data warehouse that supports multidimensional analysis
2. **Implement the HYBRIDJOIN algorithm** in Python for stream-relation join operations during the ETL transformation phase
3. **Perform comprehensive OLAP analysis** using slicing, dicing, drill-down, and materialized views
4. **Enable near-real-time data processing** to support dynamic promotions and personalized offers

1.3 System Architecture

The system consists of three main components:

- **Data Sources:** Transactional data streams and master data repositories (customer and product master data)
- **ETL Layer:** HYBRIDJOIN algorithm for extracting, transforming, and loading data with stream-relation join operations
- **Data Warehouse:** Star schema with dimension and fact tables optimized for OLAP queries

1.4 Technologies Used

- **Database:** PostgreSQL for data warehouse storage
- **Programming Language:** Python 3.x for HYBRIDJOIN implementation
- **Libraries:** pandas, psycopg2, threading, collections
- **Query Language:** SQL for schema creation and OLAP analysis

2 Data Warehouse Schema Design

2.1 Star Schema Architecture

The data warehouse follows a star schema design pattern, which consists of one central fact table surrounded by multiple dimension tables. This design enables efficient querying and intuitive multidimensional analysis.

2.2 Dimension Tables

2.2.1 dim_date (Date Dimension)

Purpose: Provides temporal context for sales transactions

Key Attributes:

- `date_key` (PK): Integer key in YYYYMMDD format
- `full_date`: Complete date value
- `day_of_week`, `month`, `quarter`, `year`: Hierarchical time components
- `is_weekend`: Boolean flag for weekend identification
- `season`: Seasonal categorization (Spring, Summer, Fall, Winter)

Design Rationale: The date dimension supports time-based analysis including weekday vs. weekend comparisons, seasonal trends, and drill-down from year to day level.

2.2.2 dim_customer (Customer Dimension)

Purpose: Stores customer demographic information

Key Attributes:

- `customer_key` (PK): Surrogate key
- `customer_id`: Business key (User_ID from source)
- `gender`, `age`, `age_group`: Demographic attributes
- `occupation`: Customer profession
- `city_category`: City classification (A, B, C)
- `stay_in_current_city_years`: Residential duration
- `marital_status`: Marital status indicator

Design Rationale: This dimension enables customer segmentation analysis by demographics, occupation, and location characteristics.

2.2.3 dim_product (Product Dimension)

Purpose: Contains product master data
Key Attributes:

- `product_key` (PK): Surrogate key
- `product_id`: Business key
- `product_category_1/2/3`: Three-level category hierarchy
- `product_name`: Product description
- `supplier_name`: Supplier information

Design Rationale: The hierarchical category structure supports drill-down analysis from broad categories to specific products.

2.2.4 dim_store (Store Dimension)

Purpose: Represents Walmart store locations
Key Attributes:

- `store_key` (PK): Surrogate key
- `store_id`: Business key
- `store_name`: Store identifier
- `store_location`: Geographic location

Design Rationale: Enables store-level performance analysis and geographic comparisons.

2.3 Fact Table

2.3.1 fact_sales (Sales Fact Table)

Purpose: Records individual sales transactions
Key Attributes:

- `sales_key` (PK): Auto-incrementing surrogate key
- `date_key` (FK): Reference to dim_date
- `customer_key` (FK): Reference to dim_customer
- `product_key` (FK): Reference to dim_product
- `store_key` (FK): Reference to dim_store
- `purchase_amount`: Transaction value (measure)
- `quantity`: Number of items sold (measure)

Design Rationale: The fact table contains foreign keys to all dimensions and measurable facts (purchase_amount, quantity) that can be aggregated for analysis.

2.4 Schema Benefits

1. **Query Performance:** Star schema design minimizes joins and improves query execution speed
2. **Intuitive Structure:** Business users can easily understand the relationships
3. **Scalability:** New dimensions or facts can be added without major restructuring
4. **Flexibility:** Supports various analytical perspectives (time, customer, product, location)

3 HYBRIDJOIN Algorithm Explanation

3.1 Algorithm Purpose

HYBRIDJOIN is a stream-based join algorithm designed specifically for near-real-time data warehousing scenarios. It efficiently joins a fast-arriving, potentially bursty data stream (S) with a large, disk-based relation (R - master data).

3.2 Key Data Structures

3.2.1 Hash Table (H)

- **Type:** Multi-map allowing multiple entries per key
- **Size:** 10,000 slots ($h_S = 10,000$)
- **Purpose:** Stores stream tuples indexed by join key (Customer_ID)
- **Structure:** Each entry contains the tuple data and a pointer to its queue node

3.2.2 Queue (Q)

- **Type:** Doubly-linked list (deque in Python)
- **Purpose:** Maintains FIFO order of stream tuple keys
- **Function:** Ensures fair processing and enables efficient random deletions

3.2.3 Stream Buffer

- **Type:** Thread-safe queue
- **Purpose:** Temporarily holds incoming stream tuples during burst periods
- **Function:** Prevents data loss when tuples arrive faster than processing speed

3.2.4 Disk Buffer

- **Type:** In-memory list
- **Size:** 500 tuples per partition ($v_P = 500$)
- **Purpose:** Holds loaded partition from master data (R)

3.3 Algorithm Workflow

3.3.1 Step 1: Initialization

```
# Initialize hash table with hS (10,000) slots
# Initialize empty queue
# Initialize stream buffer and disk buffer
# Set w = hS (available slots = 10,000)
```

Listing 1: Initialization Phase

3.3.2 Step 2: Stream Tuple Loading

```
loaded = 0
max_to_load = min(w, 1000)  # Load up to 1000 tuples at once

WHILE loaded < max_to_load AND stream_buffer NOT empty:
    - Get tuple from stream buffer
    - Extract Customer_ID as join key
    - Compute hash slot: hash(Customer_ID) % 10,000
    - Create queue node with key and tuple data
    - Add to hash table at computed slot
    - Append queue node to queue
    - Increment loaded counter

- Decrease w by loaded count (w -= loaded)
```

Listing 2: Stream Tuple Loading

3.3.3 Step 3: Disk Partition Loading

```
- Get oldest key from queue (FIFO - queue[0])
- Use key to index into master data (R)
- Filter customer master data by Customer_ID
- Load matching records into disk buffer
- Return partition size
```

Listing 3: Disk Partition Loading

3.3.4 Step 4: Probing and Joining

```
matched_keys = empty set
joined_records = empty list

FOR each tuple in disk_buffer:
    - Get Customer_ID from disk tuple
    IF Customer_ID exists in hash table:
        FOR each stream tuple with this Customer_ID:
            - Merge stream tuple with disk tuple
            - Add joined record to joined_records list
            - Add Customer_ID to matched_keys set
            - Increment matched_count

FOR each key in matched_keys:
    - Count freed_slots = number of tuples with this key
```

```
    - Remove key from hash table
    - Remove all nodes with this key from queue
    - Increase w by freed_slots (w += freed_slots)

RETURN joined_records
```

Listing 4: Probing and Joining Phase

3.3.5 Step 5: Loading to Warehouse

```
IF joined_records count >= batch_size (100):
    FOR each joined record:
        - Get or create date_key
        - Get or create customer_key
        - Get or create product_key
        - Get or create store_key
        - Extract price and quantity
        - Sanitize all values
        - INSERT into fact_sales table
        - COMMIT transaction

    - Clear batch list
```

Listing 5: Warehouse Loading

3.4 Threading Model

The implementation uses a multi-threaded approach:

Thread 1: Stream Producer

- Continuously reads transactional data from CSV file in chunks (100 records at a time)
- Adds records to stream buffer (thread-safe Queue)
- Simulates real-time data arrival with small delays (0.01s per record)
- Sets running flag to False when all data is loaded
- Runs independently of join operation

Thread 2: HYBRIDJOIN Processor (Main Thread)

- Executes the main HYBRIDJOIN algorithm
- Processes tuples from stream buffer
- Performs join operations with master data
- Loads enriched data into data warehouse
- Manages hash table, queue, and available slots (w)

3.5 Data Enrichment Process

The algorithm enriches transactional data by joining it with master data:

Input Stream Data:

- User_ID, Product_ID, Purchase amount, Date, Quantity

Master Data (Disk-based):

- Customer demographics (gender, age, occupation, city category, marital status)
- Product details (categories, name, supplier, store information, price)

Enriched Output:

- Complete customer profile + product details + transaction data
- Loaded into data warehouse dimensions and fact table with surrogate keys

3.6 Hash Function

```
def hash_function(key):
    return hash(key) % hs
```

Listing 6: Hash Function

This function maps the join key (Customer_ID) to a slot number between 0 and 9,999, ensuring even distribution across the hash table.

3.7 Type Sanitization

The implementation includes a critical `sanitize_value()` method to handle data type compatibility:

```
def sanitize_value(self, value):
    if pd.isna(value):
        return None
    if hasattr(value, 'item'):  # numpy types have .item() method
        return value.item()
    return value
```

Listing 7: Type Sanitization Method

Purpose:

- Pandas DataFrames store values as numpy types (numpy.float64, numpy.int64, etc.)
- PostgreSQL doesn't recognize numpy types and throws errors
- The `.item()` method extracts Python native types from numpy scalars
- Applied to all values before database insertion

3.8 Performance Characteristics

- **Time Complexity:** O(1) average case for hash table lookups
- **Space Complexity:** $O(h_S)$ for hash table + $O(|Q|)$ for queue + $O(v_P)$ for disk buffer
- **I/O Efficiency:** Minimizes disk accesses by using indexed master data lookups
- **Throughput:** Can handle bursty streams without data loss using stream buffer
- **Database Operations:** Individual INSERT statements with per-record commit for data integrity

4 Shortcomings of HYBRIDJOIN

While HYBRIDJOIN is effective for near-real-time data warehousing, it has several limitations:

4.1 Shortcoming 1: Limited Hash Table Size Causes Tuple Eviction

Description:
The hash table has a fixed size of 10,000 slots ($h_S = 10,000$). When the hash table fills up and no matches are found for older tuples, these unmatched tuples must be removed from both the hash table and queue to make space for new incoming tuples. This can lead to data loss if matching master data arrives later.

Impact:

- Stream tuples that arrive before their matching master data may be evicted
- Potential loss of join opportunities for late-arriving master data
- Reduced join completeness in scenarios with temporal misalignment

Example:
If a customer makes a purchase (stream tuple) but their master data is updated later in the system, the transaction tuple might be evicted before the join can occur.

Potential Solution:

- Implement a secondary storage (e.g., disk-based overflow buffer) for evicted tuples
- Use a larger hash table size if memory permits
- Implement a time-window based eviction policy
- Add a reconciliation process to re-process unmatched tuples

4.2 Shortcoming 2: Single-Attribute Join Key Limitation

Description:
HYBRIDJOIN is designed to join on a single attribute (Customer_ID in our implementation). Real-world scenarios often require multi-attribute join conditions, such as joining on both Customer_ID and Product_ID simultaneously, or joining on composite keys.

Impact:

- Cannot handle complex join predicates
- Requires multiple passes for multi-attribute joins
- Limited flexibility in join conditions
- Cannot efficiently join on conditions like "customer AND product AND date"

Example:
If we need to enrich transactions with data that depends on both customer and product combinations (e.g., customer-product affinity scores, or customer-specific product pricing), HYBRIDJOIN cannot efficiently handle this in a single pass.

Potential Solution:

- Extend the hash function to support composite keys: `hash((customer_id, product_id))`
- Implement cascading joins (first join on Customer_ID, then on Product_ID)
- Use a compound key structure in the hash table
- Implement multi-dimensional hash tables

4.3 Shortcoming 3: No Support for Late-Arriving Dimension Data

Description:
The algorithm assumes that master data (dimension data) is always available when processing stream tuples. However, in real-world scenarios, dimension data might arrive late or be updated after transactions have been processed. HYBRIDJOIN does not handle slowly changing dimensions (SCD) or late-arriving dimension updates.

Impact:

- Transactions processed before dimension updates contain outdated information
- No mechanism to retroactively update already-loaded warehouse records
- Potential for inconsistent historical analysis
- Cannot track historical changes in customer or product attributes

Example:
If a customer updates their profile (e.g., changes city from 'A' to 'B' or marital status from single to married) after several transactions have been processed, the warehouse will contain both old and new customer attributes without proper versioning. Historical analysis might incorrectly attribute pre-move purchases to the new city.

Potential Solution:

- Implement Type 2 Slowly Changing Dimension (SCD) strategy with effective dates
- Add versioning mechanism: valid_from and valid_to timestamps
- Create a reconciliation process to update historical facts when dimensions change
- Implement timestamp-based join predicates to match transaction time with dimension validity period
- Maintain dimension history tables for audit trails

4.4 Shortcoming 4: Performance Bottleneck from Individual Database Operations

Description:
The current implementation inserts records one at a time into the fact table and commits after each record. This approach, while ensuring data integrity, creates significant performance overhead due to multiple database round trips and transaction commits.

Impact:

- Each INSERT requires a separate network round trip to the database
- Each COMMIT involves transaction log writes and lock management
- Processing 100 records requires 100 INSERT operations + 100 COMMIT operations
- Throughput limited to approximately 50-100 records per second
- Scalability issues with large data volumes

Example:
Loading 10,000 transactions could take 100-200 seconds (2-3 minutes) due to individual operations, whereas batch processing could complete the same workload in 10-20 seconds.

Potential Solution:

- Implement bulk INSERT operations using `psycopg2.extras.execute_batch()`
- Batch multiple records and commit once per batch (e.g., 100 records per commit)
- Use prepared statements to reduce query parsing overhead
- Consider COPY command for very large bulk loads

4.5 Shortcoming 5: Repeated Dimension Lookups Without Caching

Description:
The implementation queries the database for dimension keys (customer_key, product_key, store_key, date_key) for every transaction, even when the same customers, products, stores, and dates appear repeatedly. This creates unnecessary database load.

Impact:

- Multiple SELECT queries for the same dimension values

- Increased database server load and network traffic
- Slower processing for transactions with duplicate dimension values
- Particularly inefficient for retail data where customers and products repeat frequently

Example:
If a customer makes 50 purchases, the system will query the database 50 times for the same customer_key instead of remembering it after the first lookup.

Potential Solution:

- Implement in-memory caching using Python dictionaries
- Cache dimension keys after first lookup: `customer_cache[customer_id] = customer_key`
- Check cache before querying database
- Clear cache periodically to handle dimension updates
- Could improve performance by 5-10x for dimension lookups

4.6 Shortcoming 6: Memory-Bound Scalability

Description:
The entire hash table and queue must fit in main memory. For very high-volume streams or scenarios with many unmatched tuples, memory constraints become a bottleneck.

Impact:

- Limited scalability for extremely high-velocity streams
- Cannot handle scenarios where unmatched tuples accumulate faster than they're processed
- Risk of out-of-memory errors in production environments

Potential Solution:

- Implement disk-based hash table spillover
- Use distributed hash tables across multiple machines
- Implement sliding window approach to limit memory usage
- Consider hybrid in-memory + disk storage strategies

5 Reflection and Lessons Learned

5.1 Technical Skills Acquired

5.1.1 Database Design

- Gained deep understanding of star schema design principles

- Learned to balance normalization with query performance
- Understood the importance of surrogate keys vs. natural keys
- Mastered foreign key relationships and referential integrity
- Learned proper indexing strategies for dimension tables

5.1.2 Algorithm Implementation

- Implemented a complex stream-processing algorithm from specification
- Learned to work with multiple concurrent data structures (hash table, queue, buffers)
- Understood trade-offs between memory usage and performance
- Developed skills in thread-safe programming for concurrent data access
- Mastered the FIFO queue pattern for fair processing

5.1.3 Python Programming

- Advanced use of Python's threading module for concurrent processing
- Efficient use of pandas for data manipulation and CSV processing
- Database connectivity using psycopg2 with connection management
- Error handling and transaction management in database operations
- Learned to handle numpy/pandas type conversion for database compatibility
- Mastered the `.item()` method for extracting native types from numpy scalars
- Implemented defaultdict for multi-map hash table structure

5.1.4 SQL Proficiency

- Advanced SQL features: window functions, CTEs, ROLLUP
- Query optimization techniques and index strategy
- Complex analytical queries with multiple aggregations
- View creation for performance optimization
- Understanding of transaction management and commit strategies

5.2 Conceptual Understanding

5.2.1 Near-Real-Time Processing

- Understood the challenges of processing streaming data
- Learned the importance of buffer management in bursty scenarios
- Recognized the trade-off between latency and throughput
- Appreciated the complexity of maintaining consistency in real-time systems
- Understood the memory-bounded nature of stream processing

5.2.2 Data Warehousing Principles

- ETL vs. ELT approaches and their applicability
- Importance of data enrichment for meaningful analysis
- Dimensional modeling for analytical workloads
- OLAP operations: slicing, dicing, drill-down, roll-up
- Trade-offs between data integrity and performance

5.2.3 Stream-Relation Joins

- Fundamentals of joining streaming data with static relations
- Memory management constraints in streaming scenarios
- FIFO queue importance for fairness and ordering
- Hash-based join techniques for performance
- Handling of unmatched tuples and eviction policies

5.3 Challenges Faced and Solutions

5.3.1 Challenge 1: Thread Synchronization

- **Problem:** Race conditions when multiple threads access the stream buffer
- **Solution:** Used Python's Queue class which provides thread-safe operations with built-in locking
- **Learning:** Importance of understanding concurrency primitives and when to use thread-safe data structures

5.3.2 Challenge 2: Memory Management

- **Problem:** Large CSV files causing memory overflow when loaded entirely into memory
- **Solution:** Implemented chunk-based reading with `pandas.read_csv(chunksize=100)`
- **Learning:** Always consider memory constraints in real-world applications; streaming processing is essential for large datasets

5.3.3 Challenge 3: Dimension Key Management

- **Problem:** Need to check if dimension records already exist before inserting duplicates
- **Solution:** Used SELECT before INSERT pattern with database queries
- **Learning:** Database indexes are crucial for lookup performance; however, repeated queries for same dimensions create overhead
- **Future Improvement:** Caching would significantly reduce database load

5.3.4 Challenge 4: Data Type Mismatches

- **Problem:** CSV data types not matching database column types; needed explicit type conversion
- **Solution:** Implemented explicit type validation, conversion, and null handling with try-catch blocks
- **Learning:** Always validate and sanitize input data; never trust external data sources

5.3.5 Challenge 5: Numpy Type Compatibility

- **Problem:** PostgreSQL rejected numpy.float64 and numpy.int64 types with error "schema 'np' does not exist"
- **Root Cause:** Pandas DataFrames store numeric values as numpy types, which PostgreSQL cannot interpret
- **Solution:** Created `sanitize_value()` method to convert all numpy/pandas types to Python native types using `.item()` method
- **Learning:** Different libraries use different type systems; always ensure type compatibility at system boundaries (Python $\leftrightarrow$ Database)
- **Critical Insight:** This was the most subtle bug - error messages were cryptic and didn't immediately reveal the type mismatch issue

5.3.6 Challenge 6: Performance Bottleneck

- **Problem:** Individual INSERT statements were slow for real-time processing requirements
- **Observation:** Processing 100 records took 2-5 seconds due to database round trips
- **Current Status:** Accepted as a limitation of the current implementation
- **Learning:** Understanding that performance optimizations (batch operations, caching) would provide significant improvements in production systems
- **Future Improvement:** Implementing execute_batch() and dimension caching could provide 10-50x speedup

5.4 Real-World Applications

This project provided insights into how large retailers like Walmart actually operate:

1. **Dynamic Pricing:** Near-real-time analysis enables price adjustments based on current demand and inventory levels
2. **Personalized Marketing:** Customer segmentation supports targeted promotions and personalized recommendations
3. **Inventory Management:** Product affinity analysis helps optimize stock levels and shelf placement
4. **Store Performance:** Comparative analysis identifies best practices and problem areas across locations
5. **Real-Time Dashboards:** Business intelligence tools can query the data warehouse for up-to-the-minute metrics

5.5 Areas for Further Improvement

5.5.1 Performance Optimization

- Implement bulk INSERT operations using `execute_batch()` for 10-50x speedup
- Add in-memory caching for dimension keys to reduce database queries by 60-90%
- Use prepared statements to reduce query parsing overhead
- Implement connection pooling for database connections

5.5.2 Scalability

- Current implementation is single-machine; distributed processing (Apache Spark, Kafka Streams) would improve scalability
- Could implement horizontal partitioning for very large fact tables
- Consider using columnar databases (e.g., Amazon Redshift, Google BigQuery) for analytical workloads

5.5.3 Data Quality

- Add comprehensive data validation and cleansing routines
- Implement data quality monitoring and alerting (e.g., detect anomalies in transaction amounts)
- Add duplicate detection and deduplication logic
- Implement referential integrity checks before loading

5.5.4 Monitoring and Observability

- Add detailed logging and monitoring of join statistics (match rate, eviction rate)
- Implement dashboard for real-time system health monitoring
- Add alerting for anomalies (e.g., sudden drop in match rate, memory usage spikes)
- Create performance metrics tracking (throughput, latency)

5.5.5 Fault Tolerance

- Implement checkpointing to recover from failures
- Add transaction rollback and retry logic for failed records
- Implement exactly-once semantics for data loading
- Add dead letter queue for problematic records

5.6 Academic vs. Industry Perspectives

5.6.1 Academic Focus

- Algorithmic correctness and theoretical properties
- Controlled datasets with known characteristics
- Emphasis on understanding underlying principles
- Clean separation of concerns
- Proof-of-concept implementations

5.6.2 Industry Reality

- Need for fault tolerance and error recovery mechanisms
- Data quality issues and edge cases (nulls, duplicates, format variations)
- Performance under real-world conditions with unpredictable loads
- Operational monitoring and maintenance requirements

- Need for documentation and knowledge transfer
- Importance of code maintainability and extensibility
- Performance optimization is critical for production deployment
- Cost considerations for database operations and infrastructure

5.6.3 Key Insight

This project bridged the gap between theory and practice. While the HYBRIDJOIN algorithm is theoretically sound, practical implementation revealed several areas where optimizations would be necessary for production use. The experience of implementing a working prototype, encountering real-world issues like type compatibility and performance bottlenecks, and identifying improvement opportunities provided invaluable learning that goes beyond theoretical algorithm study.

6 Conclusion

6.1 Project Summary

This project successfully implemented a near-real-time data warehouse for Walmart using the HYBRIDJOIN algorithm for stream-relation join operations. The implementation includes:

- A well-designed star schema with four dimension tables and one fact table
- A complete Python implementation of HYBRIDJOIN with multi-threading support
- Proper error handling, transaction management, and type conversion
- Support for comprehensive OLAP queries and business intelligence analysis
- Identification of performance bottlenecks and potential optimization strategies

6.2 Key Achievements

1. **Functional System:** Successfully loads transactional data in near-real-time while enriching it with master data
2. **Comprehensive Analysis:** Supports diverse analytical queries from product affinity to seasonal trends
3. **Scalable Design:** Architecture can be extended to handle larger data volumes and additional dimensions
4. **Robust Implementation:** Includes proper error handling, type conversion, and transaction management
5. **Best Practices:** Follows industry standards for data warehousing and ETL implementation
6. **Learning Experience:** Identified real-world challenges and potential optimization strategies

6.3 Learning Outcomes

The project provided hands-on experience with:

- Modern data warehousing architectures and design patterns
- Stream processing algorithms and their practical implementations
- Advanced SQL for business intelligence and OLAP
- Concurrent programming and thread safety considerations
- Type system compatibility between Python libraries and databases
- Real-world ETL challenges and solutions
- Performance considerations and optimization opportunities

6.4 Performance Analysis

6.4.1 Current Implementation Performance

- Processing throughput: Approximately 50-100 records per second
- Individual INSERT operations with per-record commits
- Database lookups for every dimension key (no caching)
- Suitable for small to medium datasets (¡ 50,000 records)

6.4.2 Identified Bottlenecks

1. Individual database operations (100 INSERTs instead of 1 batch)
2. Repeated dimension lookups without caching
3. Per-record commit overhead

6.4.3 Potential Improvements

- Batch operations could provide 10-50x speedup
- Dimension caching could reduce database queries by 60-90%
- Combined optimizations could enable processing of 500-1000 records/second

References

1. PostgreSQL Documentation: https://www.postgresql.org/docs/
2. Python Threading Documentation: https://docs.python.org/3/library/threading.html
3. Pandas Documentation: https://pandas.pydata.org/docs/
4. Psycopg2 Documentation: https://www.psycopg.org/docs/
5. NumPy Documentation: https://numpy.org/doc/